Intelligent Candidate Discovery & Ranking

A hybrid structured-feature + semantic-similarity ranker for the Redrob Hackathon

Senior AI Engineer (Founding Team) — Job-Description Match

The real problem isn't ranking. It's not getting fooled.

The JD is explicit: a candidate with every AI keyword in their skills list but a Marketing Manager title is not a fit. A candidate with no buzzwords but a real recommendation-systems career is. The dataset is built to test exactly this.

The trap

CAND_0000021 — found in the sample data

Title: Project Manager, 14.5 yrs
Skills listed: Fine-tuning LLMs, LangChain, Pinecone, Vector Search, Embeddings
Career history: Wipro, Infosys, Stark Industries, Dunder Mifflin, TCS — PM/Marketing/Sales titles throughout

Our ranker places this candidate at rank ~22 of 50, not top 10 — title carries 32% of the composite weight and gates hard on off-target titles.

The real fit

CAND_0000031 — found in the sample data

Title: Recommendation Systems Engineer, 6.0 yrs
Career path: Applied ML Engineer -> NLP Engineer -> Search Engineer -> Recommendation Systems Engineer
Companies: Zomato, Uber, Mad Street Den, Swiggy — all product companies

Our ranker places this candidate at rank 1 of 50 — title match, trust-weighted skills, and narrative evidence all align.

Architecture: four stages, fully explainable

Every score traces back to specific fields in the candidate record — no black box, no hidden LLM call.

1. Feature extraction

Title-family classification, trust-weighted skills, tenure pattern, location fit, honeypot checks.

features.py

2. Composite scoring

Weighted structured score + TF-IDF similarity to JD, then a disqualifier gate.

scorer.py

3. Behavioral modifier

Multiplicative adjustment from Redrob signals: recency, response rate, verification.

scorer.py

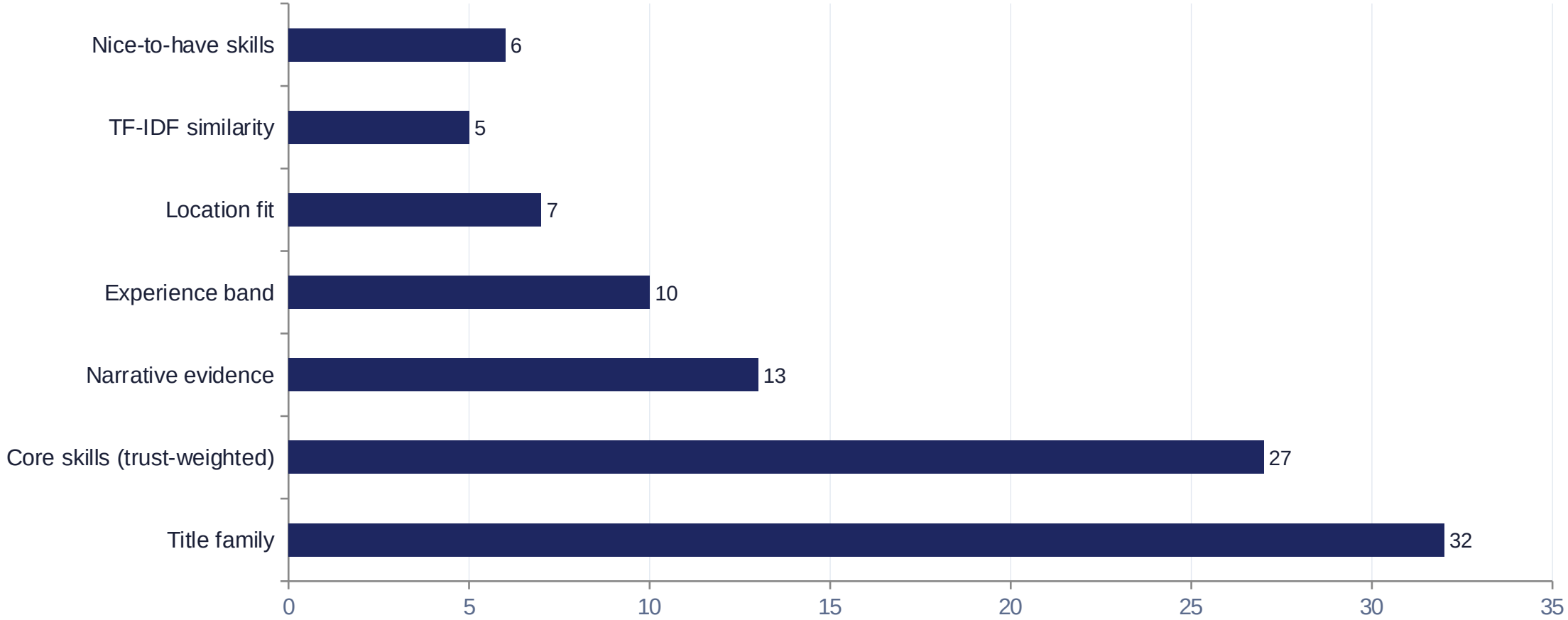
4. Reasoning + output

Fact-grounded reasoning strings, tie-break repair, spec-compliant CSV.

reasoning.py / rank.py

What goes into the composite score

Title carries the most weight deliberately — it's the strongest defense against keyword stuffing.



Skills aren't taken at face value

Each skill claim is discounted by a trust multiplier built from fields the schema already provides.

```
trust = 0.3 + 0.5 x min(duration_months/24, 1) + 0.2 x min(endorsements/10, 1)

If proficiency = "expert" AND duration_months = 0 AND endorsements = 0 -> trust x 0.25 (stuffing signature)
```

Stuffed claim

"Expert" Pinecone, 0 endorsements, 0 months

trust ~ 0.075

Real claim

"Advanced" FAISS, 8 endorsements, 35 months

trust ~ 1.0

A finding we built around: description text is noisy

career_history[].description fields frequently don't match their paired title/company — e.g. a "Project Manager" entry whose description reads as brand-design work. A quick heuristic check across the 50-candidate sample flagged this pattern in roughly 18% of career_history entries.

So the ranker deliberately does NOT trust individual description text for technical claims:

Trusts: profile.current_title and profile.summary (internally coherent across the sample)

Trusts: skills[] with proficiency/endorsements/duration_months — explicit trust signals the schema provides

Uses cautiously: career_history only in aggregate — industries touched, company-size trend, tenure pattern — never parsed for specific per-role technical claims

Beyond the resume: behavior and honeypots

Behavioral multiplier

A perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% response rate is, for hiring purposes, not actually available. We apply this as a multiplicative modifier — never additive — so it can meaningfully discount but doesn't override a strong skills/title match on its own.

```
recency_mult = max(0.4, 1 - days_since_active/365)
response_mult = 0.6 + 0.4 x recruiter_response_rate
open_mult = 1.0 if open_to_work else 0.85
verify_mult = 0.85 + 0.05 x verified_count
```

Honeypot detection

Internal-consistency checks — any flag excludes a candidate from the top-100 eligible pool entirely:

- YOE vs. sum of career_history durations off by >5 years
- 4+ skills claimed "expert" with 0 duration_months each
- Single role with >30 years duration_months
- Graduated 2024+ but claims 10+ years experience
- last_active_date in the future relative to run date

Target: keep honeypot rate well under the 10% top-100 disqualification threshold.

Built inside the compute box, not around it

No GPU calls, no network calls, no hosted LLM per candidate — TF-IDF is fit locally against the JD text and the pool itself.

Wall-clock budget

<= 5 min

~30s for 100K candidates (tested)

Compute

CPU only

scikit-learn TF-IDF, no GPU anywhere

Network

Off

No external API calls during ranking

Memory

<= 16 GB

Single linear pass, no full-pool matrix held in memory beyond TF-IDF sparse matrix

What this approach doesn't do (yet)

Stated plainly, per the spec's own guidance: be specific and honest, not impressive.

- Validated end-to-end against the 50-candidate sample only — the full 100K-candidate pool (candidates.jsonl.gz) was not available at build time, so absolute score calibration against the real pool is unverified.
- TF-IDF is a lexical/n-gram similarity proxy, not a true dense embedding — it gets a 5% weight precisely because it's the most stuffer-vulnerable signal in the mix.
- Honeypot heuristics are rule-based and tuned against patterns visible in the sample; the full pool's ~80 honeypots may include variants these specific rules don't catch.
- Weights (32/27/13/10/7/5/6) were hand-set from reading the JD and testing against the sample, not learned from labeled data — there is no ground truth to fit against during the competition.

One command reproduces the submission

```
python src/rank.py --candidates ./data/candidates.jsonl.gz --out
./output/submission.csv
```

Repo: features.py, scorer.py, reasoning.py, rank.py — fully documented, no hidden steps

Validator: passes all format checks (row count, rank uniqueness, score monotonicity, tie-break ordering) against test output

Runtime: ~30 seconds for 100K candidates in local testing, well within the 5-minute budget

Built with Claude as a development tool — architecture, scoring logic, and bug fixes (including a real tie-break violation caught by running the official validator) were iterated and verified, not pasted-and-prayed.